

Atividade 2 — Modelos incremental e evolutivo

Semana 15 · Modelos de processo · Processos de Desenvolvimento de Software

Identificação

Turma: 1º A

Data: 06/09/2026

Grupo: Grupo 3

Integrantes: Ana, Bruno, Carla, Diego

Etapa 1 — Compreensão do problema

Objetivo inicial do sistema: Permitir que o estudante registre matérias e tarefas em um app de organização de estudos.

Por que os requisitos não estão totalmente definidos: As ideias de lembretes, gráficos e calendário só surgiram depois, com o uso; o próprio time ainda não sabe como algumas telas devem funcionar.

Que tipos de mudança podem surgir com o uso: Novos tipos de lembrete, integração com o calendário da escola, relatórios para os pais, modo offline, notificações push.

Etapa 2 — Planejamento incremental

Versão 1 — o que entrega / por que nesta ordem: Cadastro de matérias e lista de tarefas com data e status. É o núcleo do produto; sem isso o app não tem valor e as outras funções dependem desses dados.

Versão 2 — o que entrega / por que nesta ordem: Lembretes automáticos das tarefas por data e horário. Agrega valor direto ao usuário e só é possível depois que as tarefas já existem (V1).

Versão 3 — o que entrega / por que nesta ordem: Gráficos de desempenho e integração com calendário. Dependem de histórico de uso acumulado e têm maior incerteza, então vêm por último.

Etapa 3 — Aplicação do modelo evolutivo

Como o feedback dos usuários gera mudanças: A cada versão entregue, os estudantes usam e apontam o que confunde ou falta; esse retorno vira ajuste de requisito para a próxima versão.

Como os requisitos são ajustados a cada ciclo: Itens do backlog são reescritos e repriorizados; requisitos vagos (“algo de gráficos”) só são detalhados quando chega a hora de construí-los.

Por que esse modelo se adapta melhor a contextos incertos: Em vez de tentar acertar tudo no papel no início, o time constrói um pouco, mostra, aprende e corrige — errando pequeno e cedo.

Etapa 4 — Relação com as fases clássicas

Explicação: As fases (requisitos, análise, projeto, implementação, testes, manutenção) não acontecem uma vez só. Cada incremento passa por um ciclo curto com todas elas: levanta-se o requisito daquele incremento, projeta-se, implementa-se, testa-se e entrega-se. A manutenção de uma versão convive com o desenvolvimento da próxima. O que muda em relação ao cascata é a repetição desses ciclos, não a existência das fases.

Etapa 5 — Diagramas no draw.io

Modelo incremental: Três blocos em sequência; cada bloco é um mini-ciclo (Requisitos → Projeto → Implementação → Testes → Entrega); uma seta geral liga Incremento 1 → 2 → 3.

Modelo evolutivo: Esboço de requisitos → Construir versão (protótipo) → Mostrar e colher retorno → losango “Atende à necessidade?”: se “Não”, volta para “Refinar requisitos e versão”; se “Sim”, segue para [Fim].

Etapa 6 — Justificativa da escolha

Justificativa: Como os requisitos do app são incertos e mudam com o uso, e o sistema pode ser dividido em partes que já entregam valor sozinhas (registrar tarefas, depois lembretes, depois gráficos), os modelos incremental e evolutivo são melhores que o cascata rígido: reduzem o risco de construir a coisa errada, entregam valor mais cedo e absorvem mudança no próximo ciclo, em vez de exigir retrabalho em cascata. Pela tabela do Capítulo 15, o cascata só seria indicado se os requisitos fossem fixos e conhecidos — o que não é o caso aqui.

Exemplo preenchido pela equipe pedagógica — serve de referência de correção. Situação fictícia produzida pela SEDUC-SP.